

KOSSCCHTHON PRD — 협업 결과물 AI 자동 정리·시각화 플랫폼

2026-09-19 · @Someone

1. 개요

한 줄 정의

팀원을 초대한 공유 스페이스에 각자 결과물을 올리면, AI가 자동으로 분석·비교해 **하나의 시각화된 정리본**으로 만들어주는 협업 플랫폼.

해결하려는 문제

서로 다른 대학·전공 학생이 협업할 때 각자의 결과물은 카톡·드라이브·노션에 흩어진다. 누가 무엇을 했는지 파악하려면 일일이 열어봐야 하고, 비슷한 고민을 한 사람이 있어도 서로 모른 채 지나간다. 결과물은 쌓이지만 **교류는 일어나지 않는다**.

대회 주제 부합 근거

주제문은 "사람과 사람, 사람과 기술 간의 교류를 확장하는 서비스"를 요구한다.

- **사람 ↔ 사람**: 내가 찾아 나서지 않아도, 같은 스페이스의 다른 전공 결과물이 내 것 옆에 자동으로 배치되어 노출됨
- **사람 ↔ 기술**: AI가 흩어진 결과물을 읽고 관계를 찾아내 시각적으로 연결 — 교류의 매개자 역할
- 교류가 '만남'이 아니라 '**결과물 간의 자동 연결**'로 일어나는 것이 이 서비스의 핵심 주장

2. 차별점 — 왜 NotebookLM/GPT로는 안 되는가

심사에서 가장 먼저 나올 질문이다. 세 가지 구조적 차이로 답한다.

	NotebookLM / GPT	본 서비스
사용 단위	1인 1세션	초대된 다수가 같은 공간
자료 수집	내가 다 모아서 넣어야 함	각자 자기 것만 올림
갱신	내가 다시 물어봐야 함	남이 올리면 내 화면이 바뀜

(1) 자료를 한 사람이 모을 필요가 없다

NotebookLM에 세 전공 결과물을 넣으려면, 한 명이 세 개를 다 확보해야 한다. 그런데 애초에 그게 안 되니까 교류가 안 되는 것이다. 본 서비스는 **각자 자기 것만 올리면 통합이 자동으로 일어난다.**

(2) 요청 없이 갱신된다

GPT는 물어봐야 답한다. 본 서비스는 다른 사람이 결과물을 올리는 순간 **내가 보고 있던 정리본이 스스로 갱신된다.** 이 '내가 요청하지 않았는데 바뀌는' 순간이 교류가 발생하는 지점이다.

(3) 공유 상태가 자산이다

대화는 끝나면 사라지지만, 스페이스는 남는다. 3개월 뒤 합류한 사람도 그동안 쌓인 관계망을 그대로 물려받는다.

3. 핵심 기능

F1. 스페이스 생성 및 초대

- 스페이스 = 하나의 주제/프로젝트 단위 작업 공간 (예: "캠퍼스 개선 아이디어")
- 생성자가 6자리 참여 코드 또는 초대 링크 발급
- 참여자는 닉네임 + 소속(학교/전공) 입력 후 입장 — 로그인 없이 진입 (해커톤 시간 절약)
- 소속 정보는 시각화에서 색상 구분에 사용 → "전공 간 교류"를 눈으로 보여주는 장치

F2. 결과물 업로드

- 입력 형태: 텍스트 직접 입력 / 파일 업로드(.txt, .md, 코드 파일)
- 필수 항목은 제목 + 본문 두 개뿐 — 진입 장벽 최소화
- 업로드 즉시 AI 분석 큐에 등록, 처리 중 상태 표시

F3. AI 자동 분석

업로드된 항목마다 두 가지를 수행한다.

1. **태깅** (gemini-2.5-flash) — 핵심 주제, 접근 방식, 사용 기술/관점을 3~5개 키워드로 추출
2. **임베딩** (gemini-embedding-001, 768차원) — 벡터 생성 후 DB 저장, 기존 항목들과 코사인 유사도 계산

F4. 그룹핑 및 비교 정리본

- 유사도 임계값(0.75 권장) 이상인 항목끼리 자동 그룹 형성

- 그룹에 2개 이상 모이면 AI가 정리본을 생성하되, 하나로 뭉뚱그리지 않고 **네 갈래로 세분화**해서 보여준다 (NotebookLM 마인드맵처럼 펼쳐보는 구조):
 - **그룹 라벨** — 예: "대기시간 문제 해결"
 - **공통점** — 모든 항목이 공유하는 핵심 문제의식 1~2줄
 - **차이점** — 항목별로 "누가 / 어떤 관점으로 / 무엇을 다르게" 접근했는지 목록
 - **비고** — 아직 아무도 다루지 않은 빈틈, 특이하게 튀는 접근, 확장 아이디어 등 부가 코멘트
- 각 갈래는 근거가 된 원본 항목으로 연결되는 "소스 N개 보기" 링크를 가짐 — 근거 없는 요약 방지

F5. 시각화 — 마인드맵 트리

정리본을 글이 아니라 그림으로 보여주는 게 이 서비스의 얼굴이다. Gemini NotebookLM의 마인드맵 결과물처럼, 스페이스 전체를 하나의 **접고 펼치는 트리**로 표현한다.

- **구조**: 루트(스페이스 주제) → 그룹(자동 클러스터) → 4개 하위 노드(공통점 / 차이점 / 비고 / 원본 목록)
- 각 노드는 화살표(>)로 접혀 있다가 클릭하면 펼쳐짐 — 처음엔 전체 구조만 한눈에 보이고, 필요한 가지만 파고들 수 있음
- 앞 노드(개별 결과물)를 클릭하면 "소스 1개 보기" 형태로 원문 패널이 열림
- 소속(학교·전공)별 색상 태그를 노드에 붙여 "전공 간 교류"가 트리 안에서 드러나게 함
- 새 그룹은 루트 아래 새 가지로, 기존 그룹 갱신은 노드에 "업데이트됨" 배지로 표시

설계 근거: 힘-방향(force-directed) 그래프 대신 트리 레이아웃을 쓰는 이유는 26시간 안에 훨씬 안정적으로 구현되고, NotebookLM을 써본 사람에게는 이미 익숙한 상호작용이라 별도 학습 비용이 없기 때문이다.

F6. 실시간 동기화

- 새 업로드 발생 → 스페이스 접속자 전원 화면에 노드 추가 (WebSocket 브로드캐스트)
- 그룹 갱신 시 해당 트리 가지만 갱신 (전체 재배포 불필요)
- 트리 구조라 그래프처럼 전체 재배포가 필요 없음 — 새 그룹은 새 가지 추가, 기존 그룹은 배지 표시로 충분

4. 사용자 플로우

1. **A가 스페이스 생성** — 주제 입력 → 참여 코드 발급
2. **B, C 입장** — 코드 입력 → 닉네임·소속 입력 → 빈 마인드맵 진입
3. **A가 첫 결과물 업로드** — B, C 화면에 노드가 즉시 나타남 (아직 그룹 없음)
4. **B가 업로드** — AI 분석 → A와 유사도 0.82 → 트리에 새 그룹 가지가 생기며 두 항목이 그 아래 묶임, 그룹 라벨 자동 생성
5. **C가 업로드** — 세 번째 항목이 같은 그룹에 합류 → 비교 정리본 자동 갱신

6. 누구나 그룹 클릭 — 공통점/차이점 패널 확인, 원문 열람

핵심 포인트는 4~5단계다. 아무도 "정리해줘"라고 요청하지 않았는데 화면이 스스로 정리되는 것이 이 서비스의 순간이다.

5. 우선순위 및 26시간 일정

기능 우선순위

기능	등급	비고
F1 스페이스·초대	P0	로그인 없이 코드 입장만
F2 업로드	P0	텍스트 입력만, 파일은 P1
F3 AI 태깅·임베딩	P0	API 호출 2회
F4 그룹핑·비교 정리본	P0	데모 하이라이트
F5 관계 지도 시각화	P0	접기/펼치기 트리로 시작
F6 실시간 동기화	P0	노드 추가 브로드캐스트만
파일 업로드	P1	시간 남으면
노드 수동 이동	P2	잘라도 무방
댓글·반응	P2	제외 권장

시간 배분 (9/19 13:00 ~ 9/20 15:00)

구간	작업
13:00~15:00	팀 빌딩, 기획 확정, 역할 분담, 스키마 설계
15:00~19:00	백엔드: 엔티티·API 골격 / 프론트: 화면 레이아웃
19:00~23:00	AI 연동(태깅·임베딩) + 유사도 계산
23:00~03:00	시각화 구현 + WebSocket 연결
03:00~07:00	통합·버그 수정
07:00~10:00	데모 데이터 준비, 리허설
10:00~15:00	발표 자료, 최종 점검, 발표

마감 규칙: 03:00 시점에 P0가 다 안 되어 있으면 시각화를 그래프에서 **그룹별 카드 목록**으로 다운그레이드한다. 완성도(50점)가 가장 큰 배점이므로 미완성 화려함보다 완성된 단순함을 택한다.

6. 기술 설계

스택

영역	선택
백엔드	Spring Boot 3, Spring Data JPA
DB	MySQL (벡터는 JSON 컬럼에 저장)
실시간	Spring WebSocket + STOMP
AI	Gemini API — 태깅·정리본: gemini-2.5-flash / 임베딩: gemini-embedding-001
프론트	React + 시각화 라이브러리

AI 벤더 선택 근거: 이 서비스는 텍스트 생성(태깅·정리본)과 임베딩(유사도 계산) 두 가지가 모두 필요하다. Anthropic은 자체 임베딩 모델을 제공하지 않고 Voyage AI를 권장하므로, Claude를 쓰려면 API 키와 SDK를 두 벌 관리해야 한다. Gemini는 생성과 임베딩을 한 벤더에서 모두 제공하므로 **Gemini 단일 벤더로 간다.**

대안으로 생성은 Claude(claude-sonnet-4-6), 임베딩은 Gemini나 Voyage를 쓰는 조합도 가능하다. 정리본 품질을 더 챙기고 싶으면 이 조합이 낫지만, 26시간 안에서는 연동 포인트가 하나 더 늘어나는 게 리스크다.

벡터DB는 쓰지 않는다. 데모 규모(수십 건)에서는 전체 조회 후 코사인 유사도를 자바로 직접 계산하는 편이 빠르고 세팅 부담도 없다. gemini-embedding-001은 출력 차원을 조절할 수 있으므로 **768차원**으로 낮춰 저장 용량과 계산량을 줄인다.

데이터 모델

엔티티	주요 필드
Space	id, name, joinCode, createdAt
Member	id, spaceId, nickname, school, major
Artifact	id, spaceId, memberId, title, content, tags, embedding, groupId

엔티티	주요 필드
Group	id, spaceId, label, commonPoints, differences, notes

API

메서드	경로	설명
POST	/api/spaces	스페이스 생성
POST	/api/spaces/{code}/join	코드로 참여
GET	/api/spaces/{id}/graph	노드·그룹 전체 조회
POST	/api/spaces/{id}/artifacts	결과물 업로드 (분석 트리거)
GET	/api/groups/{id}	비교 정리본 조회

WebSocket 이벤트

목적지	페이로드
/topic/space/{id}	ARTIFACT_ADDED — 신규 노드
/topic/space/{id}	GROUPS_UPDATED — 그룹 재계산 결과
/topic/space/{id}	MEMBER_JOINED — 참여자 입장 알림

분석 파이프라인

업로드 → 비동기 처리(@Async) → 태깅 API 호출 → 임베딩 API 호출 → 기존 항목과 유사도 계산 → 그룹 배정 → 그룹 내 2건 이상이면 비교 정리본 생성 → WebSocket 브로드캐스트

호출 방식: Spring Boot에서 Gemini는 공식 자바 SDK 대신 `RestClient` 로 REST 엔드포인트를 직접 호출하는 게 간단하다. 태깅은 `generateContent` , 임베딩은 `embedContent` 를 쓰고, 태깅 응답은 JSON만 반환하도록 프롬프트에 명시한 뒤 파싱한다.

주의 1 — 응답 지연: API 호출이 직렬로 3번이면 응답까지 5~10초 걸린다. 업로드 응답은 즉시 반환하고, 분석 완료 후 WebSocket으로 밀어주는 구조로 가야 화면이 멈추지 않는다.

주의 2 — 무료 티어 rate limit: 데모 직전 연속 업로드 테스트에서 분당 요청 제한에 걸릴 수 있다. 사전 데이터는 미리 분석해 DB에 넣어두고, 실시간 호출은 시연 1건으로 제한한다.

7. 데모 시나리오

주제: **"교내 카페 개선안"** — 같은 문제를 세 전공이 다르게 푸는 모습을 보여준다.

사전 준비

스페이스에 6~7건을 미리 넣어둔다. 빈 화면에서 시작하면 관계 지도가 안 보이고, 실시간 분석 대기 시간이 길어져 시연이 끊긴다.

시연 순서 (약 3분)

시간	장면
0:00	이미 채워진 마인드맵 제시 — 색상별 소속, 그룹 가지가 한 눈에 보임
0:30	그룹 가지 펼치기 → 공통점·차이점·비고 노드가 순서대로 드러남
1:00	화면 분할, 발표자 B가 다른 노트북으로 새 결과물 업로드
1:20	메인 화면이 저절로 갱신 — 노드 추가, 그룹 재배치, 정리본 문구 변경
2:00	해당 노드 클릭 → "이것과 가장 다른 접근" 보여주기
2:30	마무리 멘트

마무리 멘트

저희는 정리를 대신해주는 AI를 만든 게 아닙니다. 아무도 요청하지 않아도, 다른 학교·다른 전공 사람의 결과물이 제 화면에 찾아오게 만든 겁니다.

예상 질문 대비

질문	답변
NotebookLM으로 되지 않나	자료를 한 명이 다 모아야 함. 각자 올리고 자동 통합되는 게 차이
GPT에 붙여넣으면 되는데	요청해야 답하는 것과, 요청 없이 갱신되는 건 다른 경험
비슷한 협업툴 많은데	기존 툴은 저장까지. 비교·연결을 AI가 대신하는 게 핵심

8. 심사 대응 및 리스크

심사 기준 매핑

항목	배점	대응
완성도	50	P0만 확실히. 03:00 다운그레이드 규칙 준수
실용성	30	사전 데이터로 시연 안정화. 스터디·팀플·학과 협업으로 확장 서사
전달성	20	"요청 없이 갱신되는 화면" 한 장면에 집중

리스크

리스크	대응
WebSocket 처음이라 시간 소요	03:00까지 안 되면 3초 폴링으로 교체 — 화면 갱신만 보이면 데모 성립
시각화 난이도	(리스크 대폭 감소) 트리 레이아웃은 좌표 계산이 필요 없어 힘-방향 그래프보다 구현 난이도 낮음
AI 응답 지연	비동기 처리 + 사전 데이터. 실시간 분석은 데모에서 1건만
그룹핑 품질 불안정	임계값을 데모 데이터에 맞춰 사전 튜닝. 결과가 이상하면 데이터를 바꿈
팀원이 당일 배정됨	이 문서를 공유해 합의 시간 단축. 역할은 백엔드/AI/프론트/발표로 분리
Gemini 무료 티어 rate limit	사전 데이터는 미리 분석해 DB 적재, 시연 중 실시간 호출은 1건으로 제한

당일 첫 2시간 체크리스트

- 팀원 기술 스택 파악, 역할 확정
- 이 PRD 공유 후 기능 범위 합의 (P0만 하기로 명시적 동의)
- Google AI Studio에서 Gemini API 키 발급 및 호출 테스트 (무료 티어로 시작 가능)
- Git 저장소 생성, 브랜치 전략 합의
- 데모용 샘플 데이터 6~7건 문안 작성 (분석 전 미리 준비)